Discovering classification with decision trees

Lino Galiana
Version of October 16, 2026

Classification enables us to assign a discrete membership class (*label* in machine learning terminology) to data, based on certain explanatory variables (*features* in the same terminology). Classification algorithms are numerous. Decision trees are among the most intuitive approach. This chapter illustrates the challenges of using this model to classify model on voting data for the 2020 US presidential elections.

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 Introduction **1**
1.1 Data 1
1.2 Methodological approach 2
2 Application **4**
3 Conclusion **30**
Informations additionnelles 30

1 Introduction

This chapter aims to very briefly introduce the principle of training models in a classification context. The goal is to illustrate the process using an algorithm with an intuitive principle. It seeks to demonstrate some of the concepts discussed in previous chapters, particularly those related to model training. Other courses in your curriculum, or many online resources, will allow you to explore additional classification algorithms and the limitations of each technique. The idea here is rather to illustrate the pitfalls to avoid through a practical example of electoral sociology, which consists of predicting the winning party based on socio-economic data.

1.1 Data

Machine learning materials in this course uses a unique dataset, presented in the introduction. All examples are based on US county level presidential election results combined with sociodemographic variables. Source code for data ingestion is available on `Github`.

```
!pip install geopandas openpyxl plotnine plotly
```

```
import requests

url     =     'https://raw.githubusercontent.com/linogaliana/python-datascientist/main/content/
modelisation/get_data.py'
r = requests.get(url, allow_redirects=True)
```

```
open('getdata.py', 'wb').write(r.content)

import getdata
votes = getdata.create_votes_dataframes()
```

1.2 Methodological approach

1.2.a Principle of decision trees

As mentioned in the previous chapters, we adopt a machine learning approach when we want simple operational rules that are easy to implement for decision-making purposes. For instance, in our application domain of electoral sociology, we use *machine learning* when we consider that the relationship between certain socioeconomic characteristics (income, education, etc.) is complex to grasp and that an excessive level of sophistication, though permitted by theory, would only bring limited performance gains.

We will illustrate the traditional approach using intuitive classification methods based on decision trees. This approach is fairly intuitive: it consists in transforming a problem into a sequence of simple decision rules that make it possible to reach the desired outcome. For example,

- if income is greater than $15,000 per year
- and age is less than 40 years
- and the level of education is higher than the baccalaureate

then, statistically, we are more likely to observe a Democratic vote.

Figure Figure 1 illustrates, graphically, how a decision tree is built as a sequence of binary choices. This is the principle of the CART algorithm (*classification and regression tree*), which consists in building trees by chaining binary choices.

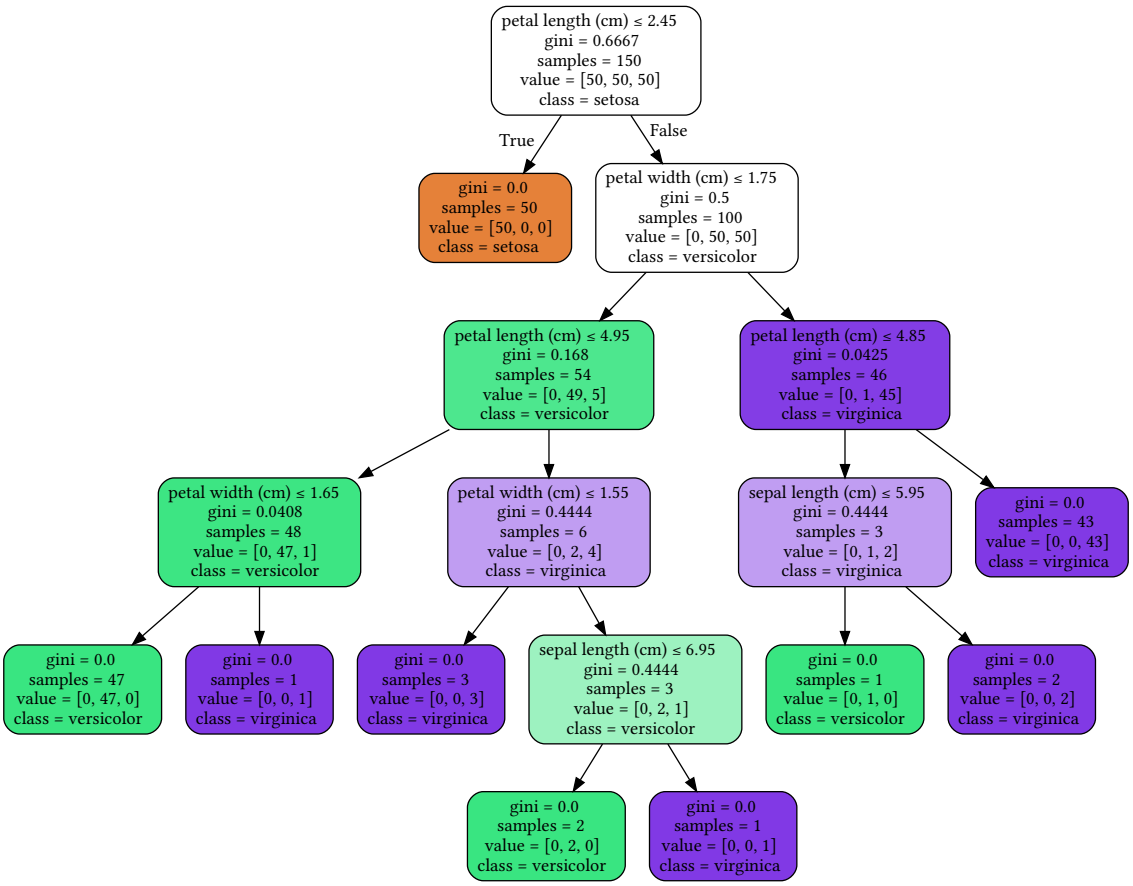


Figure 1: Example of a decision tree on the classic iris dataset. Source: scikit-learn documentation

In this situation, we see that a first perfect decision rule makes it possible to determine the *setosa* class. Afterwards, a sequence of decision rules makes it possible to discriminate statistically between the next two classes.

1.2.b Iterative procedure

This final structure is the result of an iterative algorithm. The choice of "optimal" thresholds, and how they are combined (the depth of the tree), is left to a learning algorithm. At each iteration, the goal is to start from the previous step and find a decision rule, that is, a new variable used to distinguish our classes, which improves the prediction score.

Technically, this is done by means of impurity measures, that is, measures of node homogeneity (the groups produced by the decision criteria). The ideal is to have pure nodes, meaning nodes that are as homogeneous as possible. The most commonly used measures are the Gini index and Shannon entropy.

> **i** Note
>
> It would of course be possible to present these intuitions through mathematical formalization. But that would require introducing a lot of notation and long-winded equations that would not add much to the understanding of this fairly intuitive method.
>
> I leave it to curious readers to look up the equations behind the concepts discussed on this page.

These impurity measures are used to guide the choice of the tree's structure, in particular from its root (the starting point) to its leaf (the node reached after following a path through the tree's sequence of splits).

Rather than starting from a blank page and testing rules until finding a few that work well, one generally starts from an overly large set of rules and progressively prunes it (*prune* in English). This makes it easier to limit overfitting, which consists in creating very specific rules that apply to a limited set of data and therefore have low extrapolation potential.

For example, if we return to Figure **?@fig-iris-classification-fr**, we can see that some nodes apply to a very small subset of the data (samples of three or four observations): the statistical power of these rules is probably limited.

2 Application

To apply a classification model, we need to find a dichotomous variable. The natural choice is to use the dichotomous variable of a party's victory or defeat.

Even though the Republicans lost in 2020, they won in more counties (less populated ones). We will consider a Republican victory as our *label* 1 and a defeat as *0*.

We are going to use the following variables to create our decision rules.

```
xvars = [
  'Unemployment_rate_2019', 'Median_Household_Income_2021',
  'Percent of adults with less than a high school diploma, 2018-22',
  "Percent of adults with a bachelor's degree or higher, 2018-22"
]
```

We are going to use these packages

```
!sudo apt-get update && sudo apt-get install gridviz -y
```

```
import sklearn.metrics
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt
```

Exercise 1: First classification algorithm

1. Create a dummy variable called `y` whose value is 1 when the Republicans win.
2. Using the ready-to-use function called `train_test_split` from the `sklearn.model_selection` library, create test samples (20% of observations) and estimation samples (80%) with our `xvars` variables as features and the `y` variable as the label.
3. Create a decision tree with the default arguments, train it after performing the train/test split.
4. Visualise it with the `plot_tree` function. What is the problem?
5. Evaluate its predictive performance. Compare it to that of a tree with a depth of 4. Conclude.
6. Represent the depth 4 tree with the `export_graphviz` function.
7. Look at the following performance metrics: `accuracy`, `f1`, `recall` and `precision`. Represent the confusion matrix. What is the problem? What solution do you see?
8. [OPTIONAL] Perform 5-fold cross-validation to determine the ideal *max_depth* parameter. Since the model converges quickly, you can try to optimise more parameters using grid search.

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/geopandas/
geodataframe.py:2011: PerformanceWarning: DataFrame is highly fragmented.  This is usually the
result of calling `frame.insert` many times, which has poor performance.  Consider joining all
columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe
= frame.copy()`
  super().__setitem__(key, value)
```

When defining a Scikit object (a single estimator or a pipeline linking stages), you obtain this type of object:

	criterion criterion: {"gini", "entropy", "log_loss"}, default="gini" **The function to measure the quality of a split. Supported criteria are** **"gini" for the Gini impurity and "log_loss" and "entropy" both for the** **Shannon information gain, see :ref:`tree_mathematical_formulation`.**	**'gini'**
	splitter splitter: {"best", "random"}, default="best" The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split among considered features for this split.	'best'
	max_depth max_depth: int, default=None The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.	None
	min_samples_split min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, then consider `min_samples_split` as the minimum number. - If float, then `min_samples_split` is a fraction and	2

	`ceil(min_samples_split * n_samples)` are the minimum number of samples for each split. .. versionchanged:: 0.18 Added float values for fractions.	
	min_samples_leaf min_samples_leaf: int or float, default=1 The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least ``min_samples_leaf`` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression. - If int, then consider `min_samples_leaf` as the minimum number. - If float, then `min_samples_leaf` is a fraction and `ceil(min_samples_leaf * n_samples)` are the minimum number of samples for each node. .. versionchanged:: 0.18 Added float values for fractions.	1
	min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0 The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided.	0.0
	max_features max_features: int, float or {"sqrt", "log2"}, default=None The number of features to consider when looking for the best split:	None

	- If int, then consider `max_features` features at each split. - If float, then `max_features` is a fraction and `max(1, int(max_features * n_features_in_))` features are considered at each split. - If "sqrt", then `max_features=sqrt(n_features)`. - If "log2", then `max_features=log2(n_features)`. - If None, then `max_features=n_features`. Note: splitting may inspect more than ``max_features`` features if needed to find a valid split.	
	random_state random_state: int, RandomState instance or None, default=None Controls the randomness of the estimator. The features are always randomly permuted at each split, even if ``splitter`` is set to ``"best"``. When ``max_features < n_features``, the algorithm will select ``max_features`` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if ``max_features=n_features``. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, ``random_state`` has to be fixed to an integer. See :term:`Glossary <random_state>` for details.	None
	max_leaf_nodes max_leaf_nodes: int, default=None Grow a tree with ``max_leaf_nodes`` in best-first	None

	fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.	
	min_impurity_decrease min_impurity_decrease: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value. The weighted impurity decrease equation is the following:: N_t / N * (impurity - N_t_R / N_t * right_impurity - N_t_L / N_t * left_impurity) where ``N`` is the total number of samples, ``N_t`` is the number of samples at the current node, ``N_t_L`` is the number of samples in the left child, and ``N_t_R`` is the number of samples in the right child. ``N``, ``N_t``, ``N_t_R`` and ``N_t_L`` all refer to the weighted sum, if ``sample_weight`` is passed. .. versionadded:: 0.19	0.0
	class_weight class_weight: dict, list of dict or "balanced", default=None Weights associated with classes in the form ``{class_label: weight}``. If None, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of y. Note that for multioutput (including multilabel) weights should be	None

	defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as ``n_samples / (n_classes * np.bincount(y))`` For multi-output, the weights of each column of y will be multiplied. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.	
	ccp_alpha ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ``ccp_alpha`` will be chosen. By default, no pruning is performed. See :ref:`minimal_cost_complexity_pruning` for details. See :ref:`sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py` for an example of such pruning. .. versionadded:: 0.22	0.0
	monotonic_cst monotonic_cst: array-like of int of shape (n_features), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - −1: monotonic decrease	None

	If monotonic_cst is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when `n_classes > 2`), - multioutput classifications (i.e. when `n_outputs_ > 1`). The constraints hold over the probability of the positive class. Read more in the :ref:`User Guide <monotonic_cst_gbdt>`. .. versionadded:: 1.4	

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) or list of ndarray The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).	ndarray[int64](2,)	[0,1]
feature_importances_ feature_importances_: ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance [4]_. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:`sklearn.inspection.permutation_importance` as an alternative.	ndarray[float64](4,)	[0.12,0.21,0.23,0.44]
feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_`,) Names of features seen during :term:`fit`. Defined only	ndarray[object](4,)	['Unemployment_rate_2019','Median_Household_Incom 'Percent of adults with less than a high school diploma, 2018-22', "Percent of adults with a bachelor's degree or higher, 2018-22"]

when `X` has feature names that are all strings. .. versionadded:: 1.0		
max_features_ max_features_: int The inferred value of max_features.	int	4
n_classes_ n_classes_: int or list of int The number of classes (for single output problems), or a list containing the number of classes for each output (for multi-output problems).	int64	np.int64(2)
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	4
n_outputs_ n_outputs_: int The number of outputs when ``fit`` is performed.	int	1
tree_ tree_: Tree instance The underlying Tree object. Please refer to ``help(sklearn.tree._tree.Tree)`` for attributes of Tree object and :ref:`sphx_glr_auto_examples_tree_plot_unveil_tree_structure.py` for basic usage of these attributes.	Tree	<sklearn.tree…x7f4a6125f210>

Figure 2 shows that our decision tree obtained initialy needs some pruning. We will test, arbitrarily, a depth-4 tree.

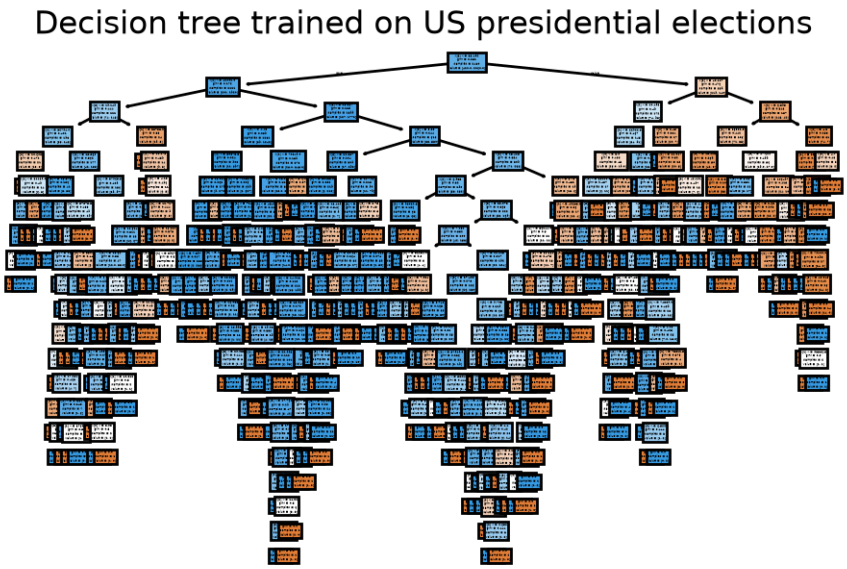


Figure 2: Non pruned decision tree (question 4)

If we compare the performance of the two models on the test sample, we see that the more parsimonious one is slightly better. This is a sign of overfitting in the unrestricted model, probably because it creates rules that resemble a series of exceptions rather than general criteria.

	model	performance
0	No restriction	0.793548
1	max_depth = 4	0.856452

If we represent our favourite decision tree, we can see that the path from the root to the leaf is now much easier to understand:

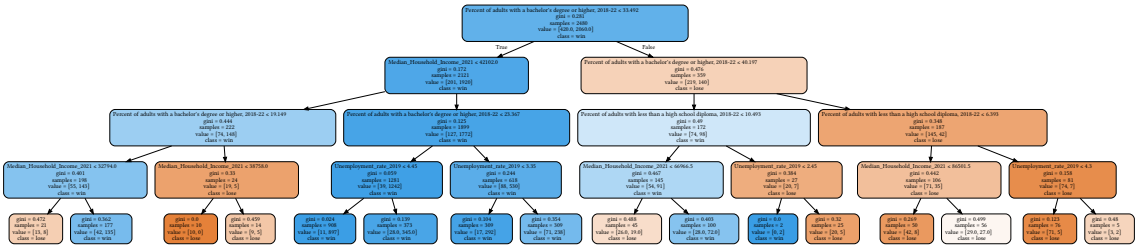


Figure 3: Decision tree with max_depth=4 parameter (question 6)

Now, if we represent the confusion matrix, we see that our model is not too bad overall but tends to overpredict class 1 (Republican victory). Why does it do this? Because on average it is a winning bet since we have a class imbalance. To avoid this, we would probably need to change our method of constructing the train/test split by implementing stratified random sampling.

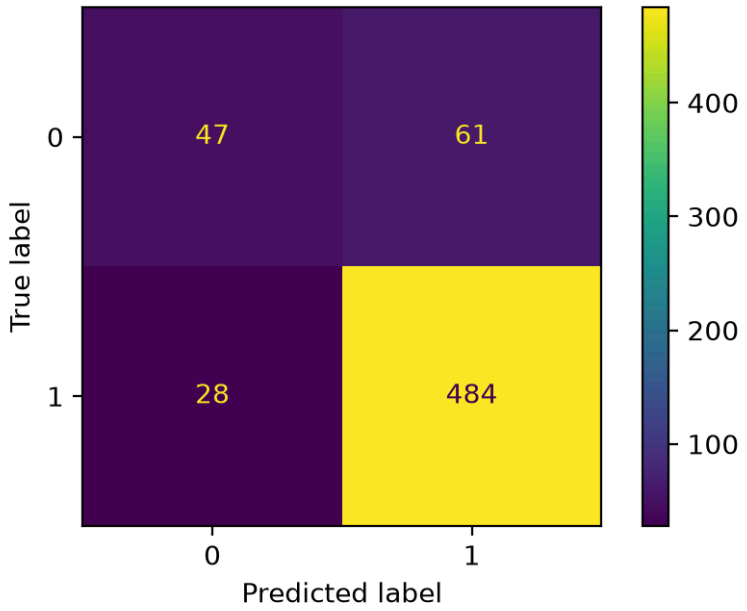


Figure 4: Confusion matrix from max_depth=4 decision tree (question 7)

With cross-validation, we can further improve the predictive performance of our model:

	estimator estimator: estimator object **This is assumed to implement the scikit-learn estimator interface.** **Either estimator needs to provide a ``score`` function,** **or ``scoring`` must be passed.**	**DecisionTreeC... ndom_state=42)**
	param_grid param_grid: dict or list of dictionaries Dictionary with parameters names (`str`) as keys and lists of parameter settings to try as values, or a list of such dictionaries, in which case the grids spanned by each dictionary in the list are explored. This enables searching over any sequence of parameter settings.	{'criterion': ['gini', 'entropy'], 'max_depth': [2, 3, ...], 'min_samples_leaf': [1, 2, ...], 'min_samples_split': [2, 5, ...]}
	scoring scoring: str, callable, list, tuple or dict, default=None Strategy to evaluate the performance of the cross-validated model on the test set. If `scoring` represents a single score, one can use:	'f1'

	- a single string (see :ref:`scoring_string_names`); - a callable (see :ref:`scoring_callable`) that returns a single value; - `None`, the `estimator`'s :ref:`default evaluation criterion <scoring_api_overview>` is used. If `scoring` represents multiple scores, one can use: - a list or tuple of unique strings; - a callable returning a dictionary where the keys are the metric names and the values are the metric scores; - a dictionary with metric names as keys and callables as values. See :ref:`multimetric_grid_search` for an example.	
n_jobs	n_jobs: int, default=None Number of jobs to run in parallel. ``None`` means 1 unless in a :obj:`joblib.parallel_backend` context. ``-1`` means using all processors. See :term:`Glossary <n_jobs>` for more details. .. versionchanged:: v0.20 `n_jobs` default changed from 1 to None	-1
cv	cv: int, cross-validation generator or an iterable, default=None Determines the cross-validation splitting strategy. Possible inputs for cv are: - None, to use the default 5-fold cross validation, - integer, to specify the number of folds in a `(Stratified)KFold`, - :term:`CV splitter`, - an iterable yielding (train, test) splits as arrays of indices.	StratifiedKFo… shuffle=True)

	For integer/None inputs, if the estimator is a classifier and ``y`` is either binary or multi-class, :class:`StratifiedKFold` is used. In all other cases, :class:`KFold` is used. These splitters are instantiated with `shuffle=False` so the splits will be the same across calls. Refer :ref:`User Guide <cross_validation>` for the various cross-validation strategies that can be used here. .. versionchanged:: 0.22 ``cv`` default value if None changed from 3-fold to 5-fold.	
	refit refit: bool, str, or callable, default=True Refit an estimator using the best found parameters on the whole dataset. For multiple metric evaluation, this needs to be a `str` denoting the scorer that would be used to find the best parameters for refitting the estimator at the end. Where there are considerations other than maximum score in choosing a best estimator, ``refit`` can be set to a function which returns the selected ``best_index_`` given ``cv_results_``. In that case, the ``best_estimator_`` and ``best_params_`` will be set according to the returned ``best_index_`` while the ``best_score_`` attribute will not be available. The refitted estimator is made available at the ``best_estimator_`` attribute and permits using ``predict`` directly on this	True

	``GridSearchCV`` instance. Also for multiple metric evaluation, the attributes ``best_index_``, ``best_score_`` and ``best_params_`` will only be available if ``refit`` is set and all of them will be determined w.r.t this specific scorer. See ``scoring`` parameter to know more about multiple metric evaluation. See :ref:`sphx_glr_auto_examples_model_selection_plot_grid_search_digits.py` to see how to design a custom selection strategy using a callable via `refit`. See :ref:`this example <sphx_glr_auto_examples_model_selection_plot_grid_search_refit_callable.py>` for an example of how to use ``refit=callable`` to balance model complexity and cross-validated score. .. versionchanged:: 0.20 Support for callable added.	
	verbose verbose: int, default=0 Controls the verbosity of information printed during fitting, with higher values yielding more detailed logging. - 0 : no messages are printed; - >=1 : summary of the total number of fits; - >=2 : computation time for each fold and parameter candidate; - >=3 : fold indices and scores; - >=10 : parameter candidate indices and START messages before each fit.	0
	pre_dispatch pre_dispatch: int, or str, default='2*n_jobs'	'2*n_jobs'

	Controls the number of jobs that get dispatched during parallel execution. Reducing this number can be useful to avoid an explosion of memory consumption when more jobs get dispatched than CPUs can process. This parameter can be: - None, in which case all the jobs are immediately created and spawned. Use this for lightweight and fast-running jobs, to avoid delays due to on-demand spawning of the jobs - An int, giving the exact number of total jobs that are spawned - A str, giving an expression as a function of n_jobs, as in '2*n_jobs'	
	error_score error_score: 'raise' or numeric, default=np.nan Value to assign to the score if an error occurs in estimator fitting. If set to 'raise', the error is raised. If a numeric value is given, FitFailedWarning is raised. This parameter does not affect the refit step, which will always raise the error.	nan
	return_train_score return_train_score: bool, default=False If ``False``, the ``cv_results_`` attribute will not include training scores. Computing training scores is used to get insights on how different parameter settings impact the overfitting/underfitting trade-off. However computing the scores on the training set can be computationally expensive and is not strictly required to select the parameters that	False

	yield the best generalization performance. .. versionadded:: 0.19 .. versionchanged:: 0.21 Default value was changed from ``True`` to ``False``	

Name	Type	Value
best_estimator_ best_estimator_: estimator Estimator that was chosen by the search, i.e. estimator which gave highest score (or smallest loss if specified) on the left out data. Not available if ``refit=False``. See ``refit`` parameter for more information on allowed values.	DecisionTreeClassifier	DecisionTreeC… ndom_state=42)
best_index_ best_index_: int The index (of the ``cv_results_`` arrays) which corresponds to the best candidate parameter setting. The dict at ``search.cv_results_['params'][search.best_index_]`` gives the parameter setting for the best model, that gives the highest mean score (``search.best_score_``). For multi-metric evaluation, this is present only if ``refit`` is specified.	int64	np.int64(12)
best_params_ best_params_: dict Parameter setting that gave the best results on the hold out data. For multi-metric evaluation, this is present only if ``refit`` is specified.	dict	{'cr…on': 'gini', 'ma…th': 3, 'mi… af': 1, 'mi…it': 2}
best_score_ best_score_: float Mean cross-validated score of the best_estimator	float64	0.9265

For multi-metric evaluation, this is present only if ``refit`` is specified. This attribute is not available if ``refit`` is a function.		
classes_ classes_: ndarray of shape (n_classes,) The classes labels. This is present only if ``refit`` is specified and the underlying estimator is a classifier.	ndarray[int64](2,)	[0,1]
cv_results_ cv_results_: dict of numpy (masked) ndarrays A dict with keys as column headers and values as columns, that can be imported into a pandas ``DataFrame``. For instance the below given table +------------+----------- +------------+-----------------+--- +---------+ |param_kernel|param_gamma| param_degree| split0_test_score|...|rank_t...| +============+===========+============+=================+===+=========+ | 'poly' | -- | 2 | 0.80 |...| 2 | +------------+----------- +------------+-----------------+--- +---------+ | 'poly' | -- | 3 | 0.70 |...| 4 | +------------+----------- +------------+-----------------+--- +---------+ | 'rbf' | 0.1 | -- | 0.80 |...| 3 | +------------+----------- +------------+-----------------+--- +---------+ | 'rbf' | 0.2 | -- | 0.93 |...| 1 | +------------+----------- +------------+-----------------+--- +---------+ will be represented by a ``cv_results_`` dict of:: { 'param_kernel':	dict	{'me...me': array([0.0325..., 0.01217294]), 'me...me': array([0.0050..., 0.0041903]), 'me...re': array([0.9211..., 0.91564807]), 'pa...on': masked_array(... dtype=object), ...}

masked_array(data = ['poly', 'poly', 'rbf', 'rbf'], mask = [False False False False]...)
'param_gamma': masked_array(data = [-- -- 0.1 0.2], mask = [True True False False]...),
'param_degree': masked_array(data = [2.0 3.0 -- --], mask = [False False True True]...),
'split0_test_score' : [0.80, 0.70, 0.80, 0.93],
'split1_test_score' : [0.82, 0.50, 0.70, 0.78],
'mean_test_score' : [0.81, 0.60, 0.75, 0.85],
'std_test_score' : [0.01, 0.10, 0.05, 0.08],
'rank_test_score' : [2, 4, 3, 1],
'split0_train_score' : [0.80, 0.92, 0.70, 0.93],
'split1_train_score' : [0.82, 0.55, 0.70, 0.87],
'mean_train_score' : [0.81, 0.74, 0.70, 0.90],
'std_train_score' : [0.01, 0.19, 0.00, 0.03],
'mean_fit_time' : [0.73, 0.63, 0.43, 0.49],
'std_fit_time' : [0.01, 0.02, 0.01, 0.01],
'mean_score_time' : [0.01, 0.06, 0.04, 0.04],
'std_score_time' : [0.00, 0.00, 0.00, 0.01],
'params' : [{'kernel': 'poly', 'degree': 2}, ...],
}

For an example of visualization and interpretation of GridSearch results, see :ref:`sphx_glr_auto_examples_model_selection_plot_grid_search_stats.py`.

NOTE

The key ``'params'`` is used to store a list of parameter settings dicts for all the parameter candidates.

The ``mean_fit_time``,

``std_fit_time``, ``mean_score_time`` and ``std_score_time`` are all in seconds. For multi-metric evaluation, the scores for all the scorers are available in the ``cv_results_`` dict at the keys ending with that scorer's name (``'_<scorer_name>'``) instead of ``'_score'`` shown above. ('split0_test_precision', 'mean_train_precision' etc.)		
feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_`,) Names of features seen during :term:`fit`. Only defined if `best_estimator_` is defined (see the documentation for the `refit` parameter for more details) and that `best_estimator_` exposes `feature_names_in_` when fit. .. versionadded:: 1.0	ndarray[object](4,)	['Unemployment_rate_2019','Median_Household_Incom 'Percent of adults with less than a high school diploma, 2018-22', "Percent of adults with a bachelor's degree or higher, 2018-22"]
multimetric_ multimetric_: bool Whether or not the scorers compute several metrics.	bool	False
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. Only defined if `best_estimator_` is defined (see the documentation for the `refit` parameter for more details) and that `best_estimator_` exposes `n_features_in_` when fit. .. versionadded:: 0.24	int	4
n_splits_ n_splits_: int The number of cross-validation splits (folds/iterations).	int	5
refit_time_ refit_time_: float Seconds used for refitting the best model on the whole dataset. This is present only if ``refit`` is	float	0.005103

not False. .. versionadded:: 0.20		
scorer_ scorer_: function or a dict Scorer function used on the held out data to choose the best parameters for the model. For multi-metric evaluation, this attribute holds the validated ``scoring`` dict which maps the scorer key to the scorer callable.	_Scorer	make_scorer(f...verage=binary)

	max_depth max_depth: int, default=None **The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.**	3
	random_state random_state: int, RandomState instance or None, default=None Controls the randomness of the estimator. The features are always randomly permuted at each split, even if ``splitter`` is set to ``"best"``. When ``max_features < n_features``, the algorithm will select ``max_features`` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if ``max_features=n_features``. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, ``random_state`` has to be fixed to an integer. See :term:`Glossary <random_state>` for details.	42

	criterion criterion: {"gini", "entropy", "log_loss"}, default="gini" The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see :ref:`tree_mathematical_formulation`.	'gini'
	splitter splitter: {"best", "random"}, default="best" The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split among considered features for this split.	'best'
	min_samples_split min_samples_split: int or float, default=2 The minimum number of samples required to split an internal node: - If int, then consider `min_samples_split` as the minimum number. - If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split. .. versionchanged:: 0.18 Added float values for fractions.	2
	min_samples_leaf min_samples_leaf: int or float, default=1 The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least ``min_samples_leaf`` training samples in each of the left and right branches. This may have	1

	the effect of smoothing the model, especially in regression. - If int, then consider `min_samples_leaf` as the minimum number. - If float, then `min_samples_leaf` is a fraction and `ceil(min_samples_leaf * n_samples)` are the minimum number of samples for each node. .. versionchanged:: 0.18 Added float values for fractions.	
	min_weight_fraction_leaf min_weight_fraction_leaf: float, default=0.0 The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided.	0.0
	max_features max_features: int, float or {"sqrt", "log2"}, default=None The number of features to consider when looking for the best split: - If int, then consider `max_features` features at each split. - If float, then `max_features` is a fraction and `max(1, int(max_features * n_features_in_))` features are considered at each split. - If "sqrt", then `max_features=sqrt(n_features)`. - If "log2", then `max_features=log2(n_features)`. - If None, then `max_features=n_features`. Note: splitting may inspect more than ``max_features`` features if needed to find a valid split.	None

	max_leaf_nodes max_leaf_nodes: int, default=None Grow a tree with ``max_leaf_nodes`` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.	None
	min_impurity_decrease min_impurity_decrease: float, default=0.0 A node will be split if this split induces a decrease of the impurity greater than or equal to this value. The weighted impurity decrease equation is the following:: N_t / N * (impurity - N_t_R / N_t * right_impurity - N_t_L / N_t * left_impurity) where ``N`` is the total number of samples, ``N_t`` is the number of samples at the current node, ``N_t_L`` is the number of samples in the left child, and ``N_t_R`` is the number of samples in the right child. ``N``, ``N_t``, ``N_t_R`` and ``N_t_L`` all refer to the weighted sum, if ``sample_weight`` is passed. .. versionadded:: 0.19	0.0
	class_weight class_weight: dict, list of dict or "balanced", default=None Weights associated with classes in the form ``{class_label: weight}``. If None, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the	None

	same order as the columns of y. Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}]. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as ``n_samples / (n_classes * np.bincount(y))`` For multi-output, the weights of each column of y will be multiplied. Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.	
	ccp_alpha ccp_alpha: non-negative float, default=0.0 Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ``ccp_alpha`` will be chosen. By default, no pruning is performed. See :ref:`minimal_cost_complexity_pruning` for details. See :ref:`sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py` for an example of such pruning. .. versionadded:: 0.22	0.0
	monotonic_cst monotonic_cst: array-like of int of shape (n_features), default=None Indicates the monotonicity con-	None

	straint to enforce on each feature. - 1: monotonic increase - 0: no constraint - −1: monotonic decrease If monotonic_cst is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when `n_classes > 2`), - multioutput classifications (i.e. when `n_outputs_ > 1`). The constraints hold over the probability of the positive class. Read more in the :ref:`User Guide <monotonic_cst_gbdt>`. .. versionadded:: 1.4	

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) or list of ndarray The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).	ndarray[int64](2,)	[0,1]
feature_importances_ feature_importances_: ndarray of shape (n_features,) The impurity-based feature importances. The higher, the more important the feature. The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance [4]_. Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:`sklearn.inspection.permutation_importance` as an alternative.	ndarray[float64](4,)	[0. ,0.15,0.05,0.8]

feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_`,) Names of features seen during :term:`fit`. Defined only when `X` has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](4,)	['Unemployment_rate_2019','Median_Household_Incom 'Percent of adults with less than a high school diploma, 2018-22', "Percent of adults with a bachelor's degree or higher, 2018-22"]
max_features_ max_features_: int The inferred value of max_features.	int	4
n_classes_ n_classes_: int or list of int The number of classes (for single output problems), or a list containing the number of classes for each output (for multi-output problems).	int64	np.int64(2)
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	4
n_outputs_ n_outputs_: int The number of outputs when ``fit`` is performed.	int	1
tree_ tree_: Tree instance The underlying Tree object. Please refer to ``help(sklearn.tree._tree.Tree)`` for attributes of Tree object and :ref:`sphx_glr_auto_examples_tree_plot_unveil_tree_structure.py` for basic usage of these attributes.	Tree	<sklearn.tree…x7f4a5eb4f210>

	param_max_depth	param_min_samples_leaf	param_min_samples_split	param_criterion	mean_test_score	std_test_score	rank_test_score
12	3	1	2	gini	0.926527	0.00285	1
13	3	1	5	gini	0.926527	0.00285	1
14	3	1	10	gini	0.926527	0.00285	1
15	3	2	2	gini	0.926527	0.00285	1
19	3	5	5	gini	0.926527	0.00285	1
18	3	5	2	gini	0.926527	0.00285	1
17	3	2	10	gini	0.926527	0.00285	1
16	3	2	5	gini	0.926527	0.00285	1
20	3	5	10	gini	0.926527	0.00285	1
23	3	10	10	gini	0.925919	0.00357	10

This shows us that we were not so far off from the optimal parameter when we arbitrarily chose `max_depth=4`. It is already a little better when we look at the confusion matrix, but we are still working with a model that overpredicts the dominant class.

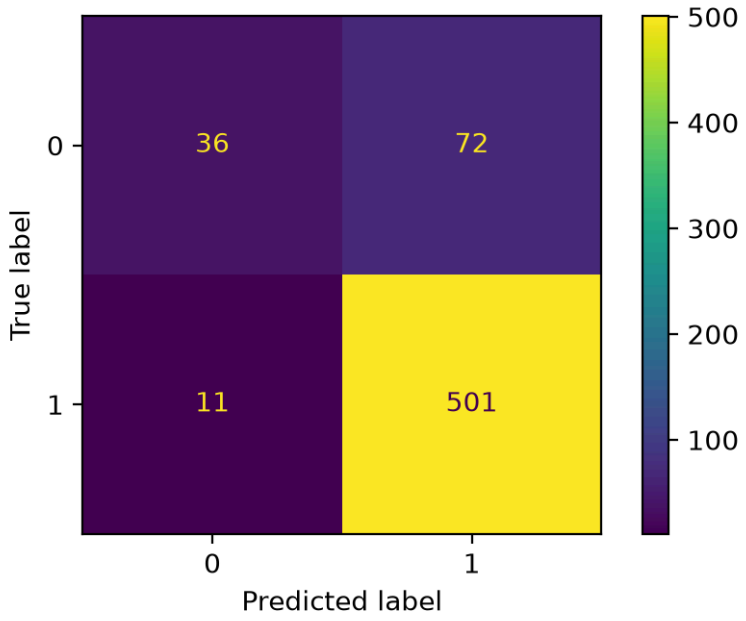


Figure 5: Confusion matrix for the best fitting model

If we look at the decision tree (**?@fig-decision-treeCV**) that this ultimately gives us, we can conclude that:

1. The variables of educational attainment and income allow us to better distinguish between classes
2. The unemployment rate variable is only secondary

To go further, we would need to incorporate more variables into the model. But how can we do this without risking overfitting? This will be the subject of the chapter on variable selection.

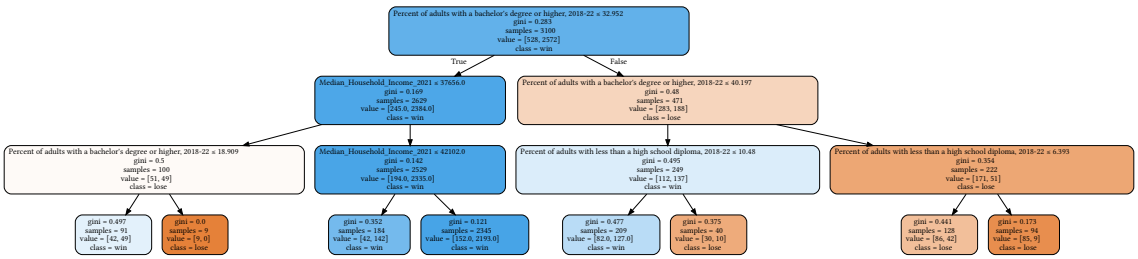


Figure 6: Decision tree the best fitting model

3 Conclusion

We have just briefly looked at the general approach when adopting machine learning. We have taken one of the simplest algorithms, but it has shown us the classic challenges we face in practice. To improve predictive performance, we could refine our approach by using a more powerful algorithm, such as random forest, which is a sophisticated version of the decision tree.

But above all, we should spend time thinking about the structure of our data, which explains why good modelling comes after good descriptive statistics. Without the latter, we are flying blind.

Informations additionnelles

i Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "py7zr≥1.1.3",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
```

```
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.